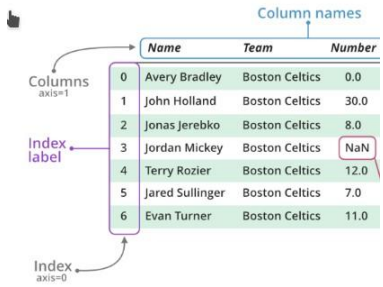
LABORATORIO 2
DESARROLLO ORIENTADO POR OBJETOS 2026-1

CONTEXTO

Objetivo: Desarrollar una versión simplificada de Pandas: BabyPandas

Pandas es una biblioteca de Python para manipulación y análisis de datos, basada en estructuras como Series y DataFrames que permiten trabajar con datos tabulares de forma eficiente. En este laboratorio desarrollaremos una versión simplificada de
Pandas con únicamente DataFrames.
Un DataFrame es una estructura de datos de dos dimensiones (rectangulares) que pueden contener datos de diferentes tipos, por lo tanto, son heterogéneas. Pandas Tutorial



ESCUELA COLOMBIANA DE INGENIERÍA JULIO GARAVITO
VIGILADA MINEDUCACIÓN
UNIVERSIDAD

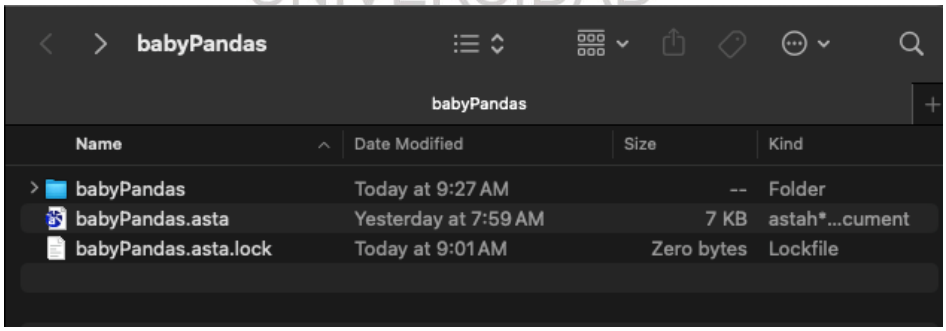
1. Conociendo el proyecto

A) Revisando línea base

[En lab02.doc]

1. **El proyecto "babyPandas" contiene una construcción parcial del sistema. Revisen el directorio donde se encuentra el proyecto. Describan el contenido en términos de**
2. **(a) directorios y de**

R/:



(b)las extensiones de los archivos.

R/:

Las extensiones de los archivos son:

- BabyPandas.class
- DataFrame.class
- DataFrameTest.class

- allclasses-index.html
- allpackages-index.html
- BabyPandas.html
- copy.html
- DataFrame.html
- DataFrameTest.html
- help-doc.html
- index.html
- index-all.html
- link.html
- overview-tree.html
- package-summary.html
- package-tree.html
- search.html
- allclasses-frame.html
- allclasses-noframe.html
- constant-values.html
- package-frame.html

ESCUELA COLOMBIANA DE INGENIERÍA JULIO GARAVITO
VIGILADA MINEDUCACIÓN
UNIVERSIDAD

3. Exploren el proyecto en BlueJ

(a) ¿Cuántas clases tiene?

R/: Tiene tres clases: BabyPandas, DataFrame, DataFrameTest.

(b) ¿Cuál es la relación entre ellas?

R/: Su relación es realizar una librería más pequeña por medio de la estructura de datos DataFrame.

(c) ¿Cuál es la clase principal de la aplicación?

R/: La clase principal de la aplicación es BabyPandas.

(d)¿Cómo la reconocen?

R/: Se reconoce porque la clase BabyPandas es la que ofrece los servicios de la aplicación.

(d) ¿Cuál es la clase "diferente"?

R/: La clase "diferente" es DataFrameTest, dado que es una clase que permite realizar casos de prueba.

(e) ¿Cuál es su propósito?

R/: Su propósito es configurar las pruebas que se realicen en la clase DataFrame y BabyPandas de manera que se pueda visualizar errores que al programar no aparecen de forma explícita.

Para las siguientes dos preguntas sólo consideren las clases "normales":

4. **Generen y revisen la documentación del proyecto: ¿está completa la documentación de cada clase? (Detallen el estado de documentación: encabezado y métodos)**
   R/: La documentación si está completa, ya que las pruebas de unidad no se documentan.

5. **Revisen el código fuente del proyecto, ¿en qué estado está cada clase? (Detallen el estado de las fuentes considerando dos dimensiones: (a) la primera, atributos y métodos, y la segunda, (b) código, documentación y comentarios) (c) ¿Qué diferencia hay entre el código, la documentación y los comentarios?**
   R/:
   (a) El estado es incompleto, ya que no se definen la mayoría de los atributos (o no se definen como en la clase DataFrame).
   (b) El estado es incompleto, debido a que algunos métodos no tienen documentado qué es lo que hace cada uno.
   (c) La diferencia radica en que el código es el ejecutable del programa (lo que hace que funcione), mientras que la documentación son explicaciones que hace cada código y la documentación es la explicación sobre cómo funciona el programa (puede ser el constructor y los métodos).

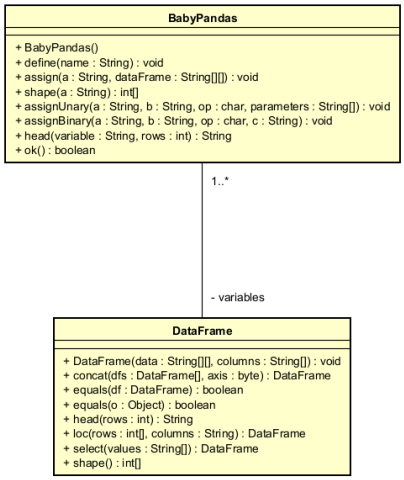
ESCUELA COLOMBIANA DE INGENIERÍA JULIO GARAVITO
VIGILADA MINEDUCACIÓN

B) Realizando ingeniería inversa

[En lab02.doc babyPandas.astah]

MDD: MODEL DRIVEN DEVELOPMENT

1. **Completen el diagrama de clases correspondiente al proyecto. (No incluyan la clase de pruebas)**



2. **(a) ¿Cuáles contenedores están definidos?**
   R/: HashMap, String[], String[][], int[], Dataframe[].

(b) ¿Qué diferencias hay entre el nuevo contenedor y el ArrayList y el vector [] que conocemos? Consulte el API de java.
R/: Una ArrayList permite listas dinámicas sin tener un tamaño fijo al crearse, mientras que un vector contiene un índice entero que accede a los componentes donde el tamaño puede aumentar o disminuir si es necesario. Por último, un HashMap asocia claves con valores para acceder eficazmente a ellos, pero no garantiza el orden del mapa.

3. **En el nuevo contenedor,**

(a) ¿Cómo adicionamos un elemento?

R/: Para adicionar un elemento en HashMap se utiliza put(clave, valor).

Clave: Lo que identifica al elemento.

(b) ¿Cómo lo consultamos?

R/: Para consultar un elemento en HashMap se usa get(clave).

(c) ¿Cómo lo eliminamos?

R/: Para eliminar un elemento en HashMap se utiliza remove(clave, valor).

ESCUELA COLOMBIANA DE INGENIERÍA JULIO GARAVITO
VIGILADA MINEDUCACIÓN
UNIVERSIDAD

2. Aprendiendo a probar

A) Conociendo Pruebas en BlueJ

[En lab02.doc *.java]

BDD: BEHAVIOUR DRIVEN DEVELOPMENT

Para poder cumplir con la prácticas XP vamos a aprender a realizar las pruebas de unidad usando las herramientas apropiadas. Para eso implementaremos algunos métodos en la clase DataFrame.

1. **Revisen el código de la clase DataFrameTest(a) ¿cuáles etiquetas tiene (componentes con símbolo @)? (b) ¿cuántos métodos tiene? (c) ¿cuantos métodos son de prueba? (d) ¿cómo los reconocen?**

R/:
(a) Tiene 5 etiquetas.
(b) Tiene 5 métodos.
(c) Hay 3 métodos de prueba.
(d) Se reconocen porque hay métodos que contienen @Test.

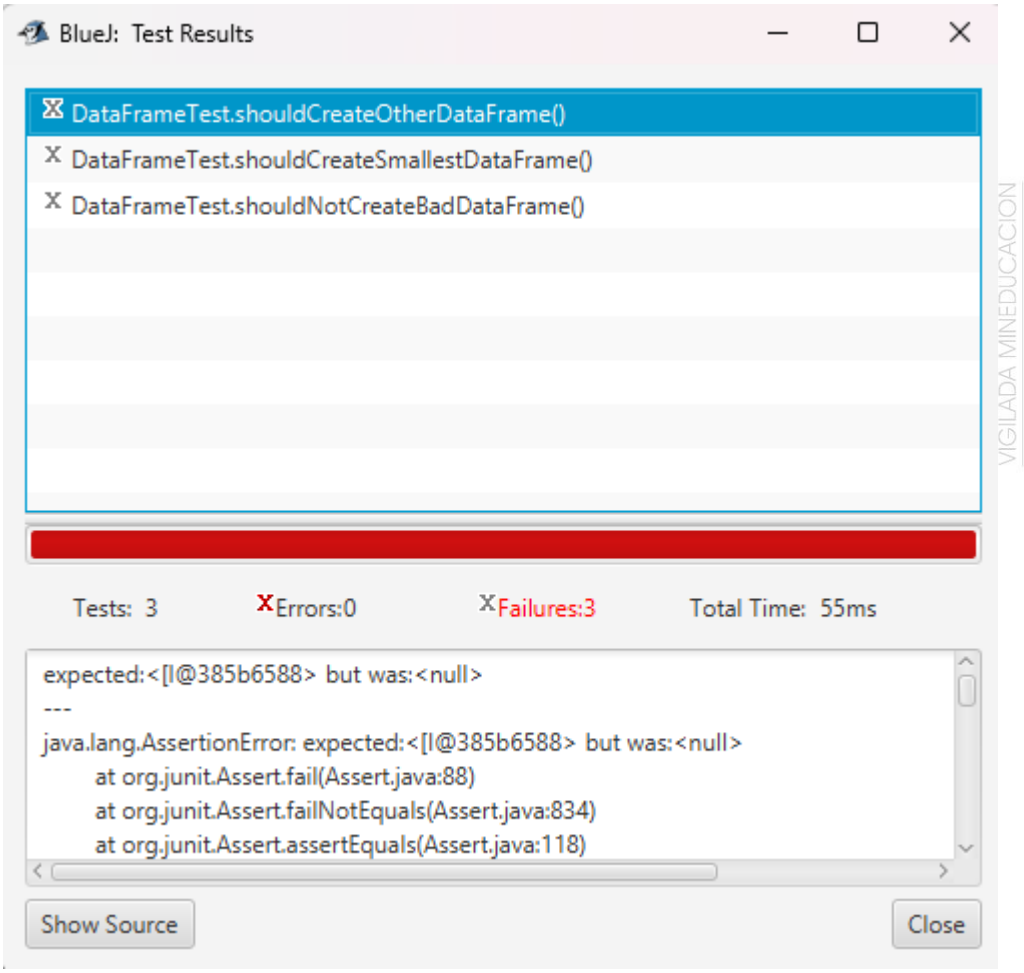
2. **Ejecuten los tests de la clase DataFrameTest. (click derecho sobre la clase, Test All)**
   (a) ¿cuántas pruebas se ejecutan?

R/: Se ejecutan tres pruebas.

(b) ¿cuántas pasan?
R/: No pasa ninguna.

(c) ¿por qué?
R/: Porque no se han definido métodos en la clase DataFrame para llevar a cabo la instanciación de un objeto (van a ser nulas).

(d) Capturen la pantalla.
R/:



VIGILADA MINEDUCACION

3. **Estudien las etiquetas encontradas en 1. Expliquen en sus palabras su significado.**
R/:
@Before: indica que un método debe ejecutarse antes de un caso de prueba.
@Test: sirve para identificar que un método es un caso de prueba.
@After: funciona para decir que un método debe ejecutarse después de los casos de prueba.

4. **Estudie los métodos assertTrue, assertFalse, assertEquals, assertNull y fail de la clase Assert del API JUnit 1. Explique en sus palabras que hace cada uno de ellos.**
   R/:
   assertTrue: se utiliza para verificar que una condición booleana es verdadera durante la prueba.
   assertFalse: se usa para verificar que una condición booleana es falsa durante la prueba. Si es falsa, no pasa la prueba.
   assertEquals: es usado para verificar que dos condiciones sean equivalentes durante una prueba.
   assertNull: se utiliza para verificar que una condición sea nula. Si es nula, entonces falla la prueba.
   fail: se usa con el fin de fallar explícitamente una prueba si esta no cumple ciertas condiciones.

5. **Investiguen y expliquen la diferencia que hay entre un fallo y un error en Junit. Escriba código, usando los métodos del punto 4., para codificar los siguientes tres casos de prueba y lograr que se comporten como lo prometen shouldPass, shouldFail, shouldErr.**

   R/: En Junit, un fallo significa que el código ejecuta, pero no se obtiene el resultado que se espera mientras que un error sucede porque hubo un problema técnico o una excepción que no fue controlada.

ESCUELA COLOMBIANA DE INGENIERÍA JULIO GARAVITO
VIGILADA MINEDUCACIÓN

B) Practicando Pruebas en BlueJ

[En lab02.doc, DataFrame, *.java]

BDD: BEHAVIOUR DRIVEN DEVELOPMENT

Ahora vamos a escribir el código necesario para que las pruebas de DataFrameTest pasen.

1. **(a) Determinen los atributos de la clase DataFrame.**
   R/:

```
public class DataFrame {
    private String[][] data;
    private String[] columns;
    private int[] shape;
```

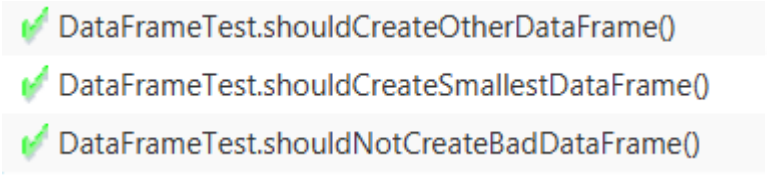
   (b) Justifiquen la selección.
   R/: Se utilizaron estos atributos con el fin de definir las filas y columnas de la estructura de datos DataFrame. Así mismo, se determinó un atributo para indicar el tamaño de BabyPandas, es decir, estas trabajan con la longitud que tenga el vector data y la longitud que tenga el vector columns.

2. **(a) Determinen el invariante de la clase DataFrame.**
   R/:

```
// Invariante
// data.length == columns.length
// shape[0] >= 0
// shape[1] > 0
```

3. **(b) Justifiquen la decisión.**
   R/: Consideramos que la invariante de la clase son estas condiciones, ya que, para crear una tabla, sus filas y columnas deben ser del mismo tamaño. Así mismo, se tiene en cuenta que no puede haber columnas, pero sí debe haber columnas como mínimo.

4. **Implementen los métodos de DataFrame necesarios para pasar todas las pruebas definidas.**
   (a) ¿Cuáles métodos implementaron?
   R/: Se implementó un método shapes. Esto es debido a que lo que hace es tener la cantidad de filas que sean válidas (mismo tamaño que las columnas), es decir, helper y tener el tamaño de las columnas para que cuando se llame a la clase DataFrame entonces se pueda verificar que los vectores sean iguales.
5. **Capturen los resultados de las pruebas de unidad.**



3. Desarrollando el proyecto babyPandas.

[En lab02.doc, desi, *.java] **BDD - MDD Para desarrollar esta aplicación vamos a considerar algunos ciclos.**

En cada ciclo deben realizar los pasos definidos a continuación.

I **Definir los métodos base de correspondientes al mini-ciclo actual.**
II **Definir y programar los casos de prueba de esos métodos.**
   Piensen en los deberia y los noDeberia (should... y shouldNot...)
III **Diseñar los métodos**
   Usen diagramas de secuencia. En astah, creen el diagrama sobre el método correspondiente.
IV **Escribir el código correspondiente (no olvide la documentación)**
V **Ejecutar las pruebas de unidad (vuelva a 3 (a veces a 2), si no están en verde)**
VI **Completar la tabla de clases y métodos. (Al final del documento)**

Ciclo 1 : Operaciones básicas: definir una variable, asignar a una variable un DataFrame, consultar el tamaño, consultar las primeras filas(head).

Ciclo 2: Operaciones unarias: seleccionar filas y columnas, seleccionar las filas que cumplen una condición.

Ciclo 3 : Operaciones binarias: concatenar por filas y columnas.

Completen la siguiente tabla indicando el número de ciclo y los métodos asociados de cada clase.

Ciclo	BabyPandas	BabyPandasTest

COLEGIO MAYOR DE INGENIERÍA JULIO GARAVITO UNIVERSIDAD VILLADA MINE

1	• define • assign • shape • head	• shouldHaveAName • shouldNotNameNull • shouldNotAssignNull
2	• assignUnary	• shouldSelectRowsCorrectly • shouldSelectColumnsCorrectly • shouldFilterByConditionCorrectly
3	• assignBinary	• shouldSelectRowsCorrectly • shouldSelectColumnsCorrectly • shouldFilterByConditionCorrectly

RETROSPECTIVA

1. **¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes? (Horas)**
   R/: El tiempo total invertido de ambos fue de 144 horas.

2. **¿Cuál es el estado actual del laboratorio? ¿Por qué?**
   R/: Completado, ya que pudimos realizar gran parte de lo que pedía el laboratorio.

3. **Considerando las prácticas XP del laboratorio, ¿cuál fue la más útil? ¿por qué?**
   R/: La práctica más útil fue "Testing. All codes must have unit tests", dado que nos ayudó a corregir errores que, al instanciar un objeto, nos generaban; por lo que crear casos de prueba ayuda demasiado a la hora de tratar de programar una aplicación que pueda funcionar.

4. **¿Cuál consideran fue el mayor logro? ¿Por qué?**
   R/: El mayor logro que consideramos fue entender un poco más lo que es el contenedor HashMap, cómo tratar de aplicarlo y cómo aplicar la clase StringBuilder.

5. **¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?**
   R/:
   No hubo ningún problema técnico durante el desarrollo del laboratorio.

6. **¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?**
   R/: Comunicarnos y apoyarnos en todo momento, habiendo una mejora a comparación con el laboratorio anterior donde era poca la comunicación.

7. **¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.**
   R/:

- W3Schools. *W3Schools — Tutoriales y referencias de programación*. https://www.w3schools.com/
- Oracle. (2015). *Class String Builder* (JavaPlatform SE8). https://docs.oracle.com/javase/8/docs/api/java/lang/StringBuilder.html

ESCUELA COLOMBIANA DE INGENIERÍA JULIO GARAVITO UNIVERSIDAD